



Instituto Superior de Engenharia

Politécnico de Coimbra

DEPARTAMENTO DE ENGENHARIA INFORMÁTICA

Trabalho Prático em React - “Minesweeper”

Autores

Hugo Rafael da Assunção Gomes – 2023133155

Rita Mariana Alves Henriques – 2023133628

Tiago Gouveia Filipe – 2019112767

Curso

Licenciatura em Engenharia Informática

Docente responsável:

Cristiana Manuela Afonso Areias



INSTITUTO POLITÉCNICO
DE COIMBRA

INSTITUTO SUPERIOR
DE ENGENHARIA
DE COIMBRA

Coimbra, junho 2024

ÍNDICE

1.	Introdução	2
2.	Componentes	2
2.1.	App	4
2.1.1.	Header.....	4
2.1.2.	Control-panel	5
2.1.2.1.	Remaining	5
2.1.2.2.	Time.....	5
2.1.3.	Game-panel	5
2.1.3.1.	Square.....	6
2.1.4.	Footer.....	6
3.	Funções auxiliares.....	7
3.1.	Função shuffleArray	7
3.2.	Função finalArrayGenerator.....	7
3.3.	Função checkNeighbours	7
4.	Limitações e desafios.....	8
5.	Conclusão	8

1. INTRODUÇÃO

No âmbito da disciplina de Linguagens Script, foi-nos proposto desenvolver o tradicional jogo Minesweeper, cujo objetivo é encontrar todas as bombas existentes numa área de jogo. O jogo é constituído por um tabuleiro com várias quadrículas com bombas escondidas, terminando quando todas as bombas forem encontradas ou quando uma bomba for seleccionada, explodindo o tabuleiro e mostrando onde todas as restantes bombas se encontravam.

A aplicação proposta foi desenvolvida em React JS, recorrendo igualmente aos conhecimentos prévios das tecnologias de HTML, CSS e JavaScript. O presente relatório pretende clarificar os componentes criados, bem como as funcionalidades implementadas.

2. COMPONENTES

Foram criados sete componentes, para além do “App”, no presente projeto, sendo apresentados de seguida. Todos os componentes são funcionais.

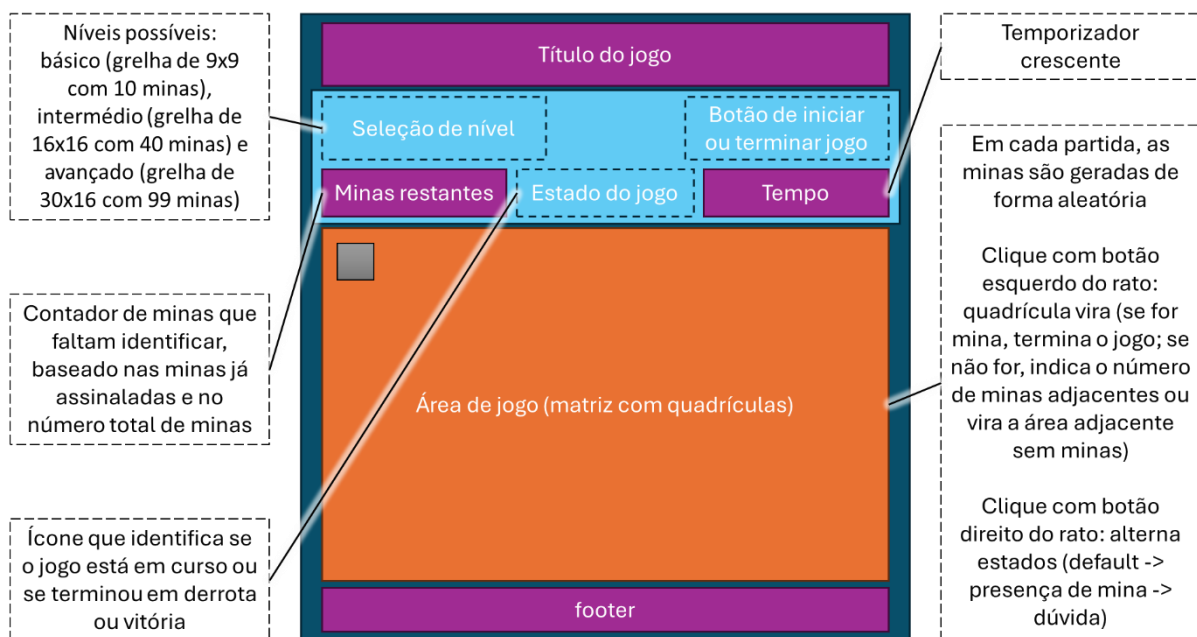


Fig. 1 Esquema da aplicação desenvolvida, com as funcionalidades desejadas para cada componente. As funcionalidades dos componentes autoexplicativos não se encontram descritas.

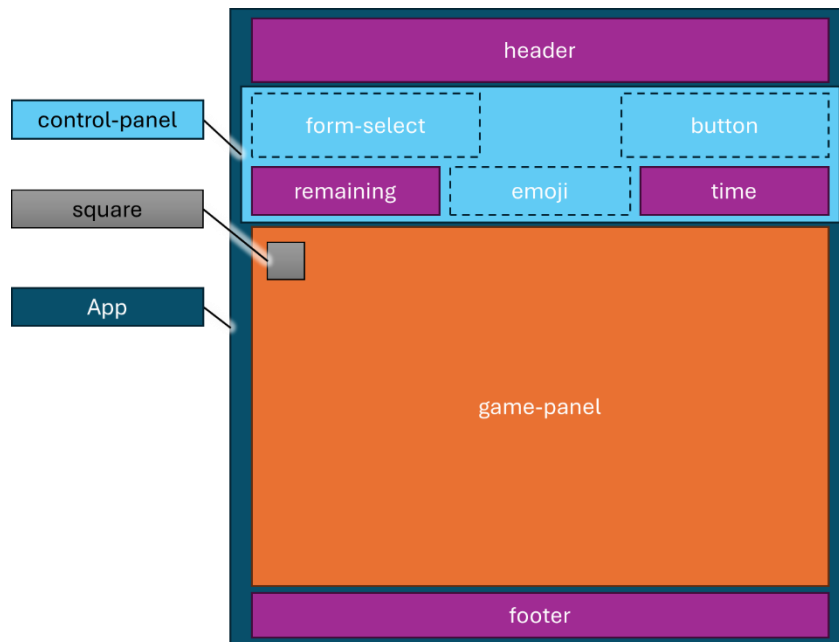


Fig. 2 Diagrama de componentes, com os respetivos nomes, utilizados na aplicação desenvolvida. De notar que os retângulos que surgem a tracejado não se tratam de componentes isolados, mas sim elementos HTML incluídos no componente “Control-panel”.

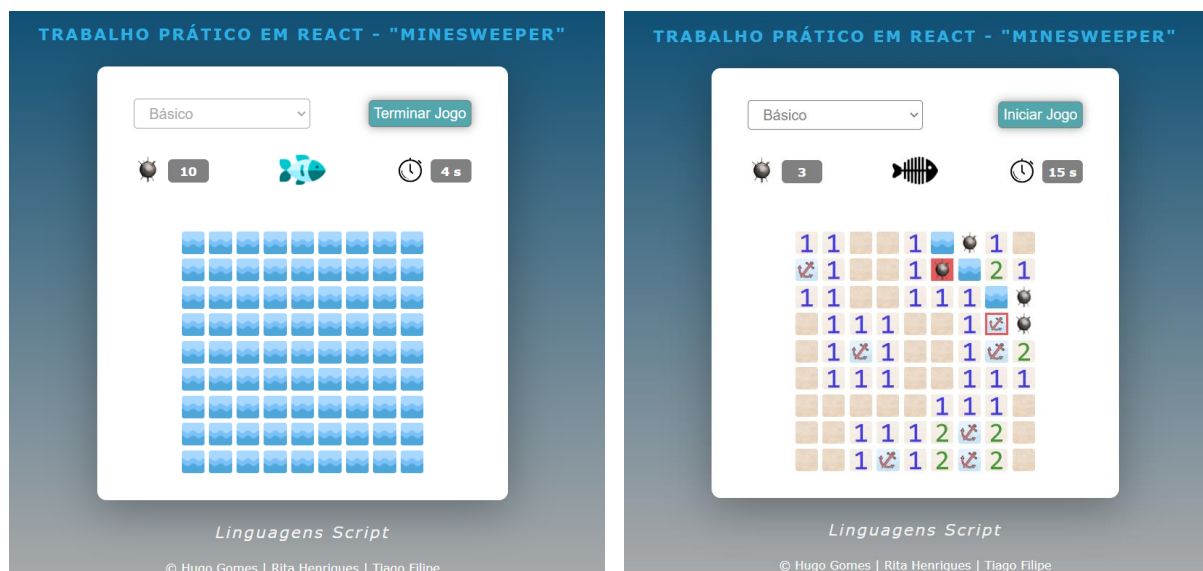


Fig. 3 Interface do jogo desenvolvido.

2.1. App

A “App”, o componente principal ou root da aplicação, comumente presente neste tipo de projetos, permite a passagem de informação entre todos os seus componentes filhos. Neste projeto, decidiu-se que a criação do mapa de jogo seria implementada neste componente, sendo guardado na variável de estado “squaresArray”. De seguida, será feita uma breve explicação das principais funções implementadas neste componente.

A função “createPanel” é chamada sempre que o nível de dificuldade é alterado ou quando se clica para iniciar um novo jogo, sendo responsável pela criação da nova matriz utilizada para o criar o mapa de jogo.

A função “handleBombsFound” atualiza a variável de estado “bombsFound” sempre que o utilizador coloca/remove bandeiras, sendo chamada no componente “Square”. Esta variável de estado permite mostrar ao utilizador o número de bombas por descobrir, sendo utilizada no componente “Remaining”.

A função “handleGameStart” é chamada sempre que o utilizador clica no botão de iniciar/terminar jogo, existente no componente “Control-panel”. Ela verifica se existe um jogo a decorrer, através da variável de estado “gameStart”, e inicia ou termina um jogo com base no estado atual da mesma. Atualiza ainda as variáveis de estado necessárias para uma correta renderização da aplicação, criando um mapa de jogo, através da chamada à função “createPanel”.

A função “handleLevelChange” é chamada sempre que o utilizador seleciona um nível de dificuldade no menu drop-down, existente no componente “Control-panel”. Atualiza as variáveis de estado necessárias para uma correta criação do novo mapa de jogo, através da chamada à função “createPanel”.

A função “handleGameOver” é chamada no componente “Game-Panel” sempre que se verificam as condições de fim de jogo. Ela atualiza as variáveis de estado necessárias para uma correta renderização da aplicação, conforme o utilizador ganhou ou perdeu o jogo.

2.1.1. Header

O componente “Header” é montado a partir do componente “App” e insere o nome do presente trabalho prático no cabeçalho da página web.

2.1.2. Control-panel

O componente “Control-panel” é filho do componente “App” e possui todas as informações relativas ao jogo. Neste componente, podemos alterar o nível de dificuldade, iniciar e terminar um jogo, bem como visualizar um contador de bombas, que inicialmente indica o número de bombas do nível em questão, um indicador de tempo decorrido, em segundos, e uma imagem de um peixe, que altera consoante o estado de jogo (por iniciar/em curso, vitória e derrota).

2.1.2.1. Remaining

O componente “Remaining” é filho do componente “Control-panel”. É o componente que indica ao utilizador quantas bombas existem por encontrar. Inicia sempre com o total de bombas do nível correspondente e, à medida que o utilizador coloca ou remove bandeiras, este contador diminui ou aumenta, respetivamente, com base na variável de estado “bombsFound” criada no componente “App” e passada por props.

2.1.2.2. Time

O componente “Time” é também filho do componente “Control-panel”, sendo utilizado para a contagem do tempo decorrido sempre que um jogo se inicia. Assim que um jogo termina, seja em vitória ou derrota, o tempo decorrido é fixado, mostrando o tempo total de jogo.

O uso deste componente deve-se ao facto de haver a necessidade de renderizações consecutivas, a cada segundo, que não influenciam os restantes componentes, não havendo a necessidade de os renderizar. Isto é conseguido com a utilização conjunta de “Hooks” e do método `setInterval()`, para que, a cada segundo, seja atualizada uma variável de estado. Incrementando-a, o componente é continuamente renderizado.

2.1.3. Game-panel

O componente “Game-panel” é filho do “App”, sendo responsável pela área de jogo. Neste componente é definido o tamanho do mapa de jogo através de classes CSS, com base no nível selecionado. Existem várias funções importantes, descritas de seguida.

A função “handleFlip” verifica que quadrícula foi clicada, sendo chamada sempre que uma quadrícula é revelada no seu componente filho “Square”. No caso de ter sido revelada uma bomba, altera a propriedade “isFlipped” de todas as bombas para

2, de forma a revelá-las no fim de jogo, e chama a função “handleGameOver”, passando como argumento o valor “2”, que significa fim de jogo por derrota. Caso tenha sido revelada uma quadrícula vazia ou um número, altera a propriedade “isFlipped” para 1 e chama a função “countFlipped”. Ainda no caso de ser uma quadrícula vazia, chama a função auxiliar “checkNeighbours”, explicada no capítulo 3.2, para revelar as quadrículas vizinhas e atualiza uma variável de estado, por forma a renderizar todo o mapa de jogo.

A função “countFlipped” verifica se o jogo terminou, calculando o número de quadrículas que possuem a propriedade “isFlipped” a 1, ou seja, que estão reveladas. Se o número de quadrículas reveladas for igual à diferença entre o tamanho total da matriz e o número de bombas correspondentes ao nível em jogo, significa que o jogo termina em vitória, chamando a função “handleGameOver” passando como argumento o valor “1”.

2.1.3.1. Square

O componente “Square” é o responsável por toda a lógica que envolve as quadrículas da área de jogo. Por forma a mostrar corretamente o seu conteúdo nos diferentes casos possíveis, são utilizadas imagens pré-definidas e classes CSS, sendo a renderização deste componente realizada através de variáveis de estado. Este componente possui duas funções principais:

- A função “handleLeftClick”, chamada quando se clica com o botão esquerdo do rato, revela o conteúdo da quadrícula e chama a função “handleFlip”, explicada no componente “Game-panel”.
- A função “handleRightClick”, chamada quando se clica com o botão direito do rato, permite a alternância entre três estados sequenciais – inserir uma bandeira, um ponto de interrogação ou voltar ao estado inicial de quadrícula escondida.

Existe ainda um “Hook” do tipo `useEffect()`, que força a renderização correta das quadrículas no início de um jogo.

2.1.4. Footer

O componente “Footer” é montado a partir do componente “App” e insere o nome da unidade curricular e dos autores do presente trabalho no rodapé da página web.

3. FUNÇÕES AUXILIARES

Foram criadas três funções auxiliares no presente projeto, sendo apresentadas de seguida.

3.1. Função shuffleArray

A função “shuffleArray”, presente no ficheiro “shuffle”, recebe do componente “App” um array unidimensional cujas bombas se encontram apenas nas posições iniciais. Esta função auxiliar foi criada exclusivamente para baralhar os elementos deste array.

3.2. Função finalArrayGenerator

A função “finalArrayGenerator”, presente no ficheiro “2d-array”, recebe do componente “App” um array unidimensional e o nível selecionado. O array já contém todos os elementos que irão fazer parte da área de jogo, inclusive as bombas em posições aleatórias. Esta função auxiliar foi criada para converter o array numa matriz (nested array), bem como para editar a propriedade “name” dos elementos que não são bombas. Assim, para cada elemento da matriz, no caso de não ser bomba, é contabilizado o número de bombas adjacentes e atribuído esse valor à propriedade “name”. Tendo os elementos na forma matricial, tornam-se mais simples as condições de pesquisa dentro da área de jogo.

3.3. Função checkNeighbours

A função “checkNeighbours”, presente no ficheiro com o mesmo nome, recebe do componente “Game-panel” uma matriz, as dimensões da área de jogo – largura e altura – e os índices que identificam a linha e coluna – “i” e “j” – de um elemento da matriz cujas vizinhanças se pretendem verificar. A verificação consiste em analisar a propriedade “name” dos elementos vizinhos. Caso essa propriedade seja 0, a propriedade “isFlipped” passa a ser 1 e a função chama-se a si própria através de um comportamento recursivo; caso o “name” seja diferente de 0, apenas se altera a propriedade “isFlipped” para 1. Esta propriedade servirá para virar as quadrículas de uma área adjacente sem bombas, delimitada apenas por números.

4. LIMITAÇÕES E DESAFIOS

O maior desafio encontrado está relacionado com o reset das partidas. Por vezes, quando se iniciava um novo jogo após já se ter jogado anteriormente, existiam problemas de renderização do novo mapa de jogo, onde quadrículas já se encontravam reveladas, com bandeiras ou pontos de interrogação. Uma vez que as variáveis de estado que assinalam o fim de jogo já são usados para outras funcionalidades no componente “Square”, tivemos dificuldade em encontrar condições para recorrer a essas variáveis e forçar a viragem das cartas antes de um novo jogo se iniciar. Por forma a garantir a correta renderização, foi criada a variável de estado “forceTurn” no componente “App”, que é passada através de props até ao componente “Square”, onde força a correta renderização das quadrículas, com base nas classes CSS e variáveis de estado aí definidas.

A única limitação encontrada neste projeto prende-se com o facto de não se ter conseguido renderizar apenas o componente “Remaining”, sempre que o utilizador coloca e remove bandeiras, sendo essa informação dada pelo componente “Square”. Deste modo, a variável de estado usada para o efeito foi criada no “App” e passada através de props até aos componentes necessários, tendo-se a consciência que se está a renderizar toda a página sempre que há alteração. Poderíamos ter criado um “Hook” do tipo `useEffect()` no componente “Remaining” que detetasse alterações do número de bombas assinaladas com bandeiras, mas tal só atualizaria se houvesse uma alteração de uma variável de estado no componente “App”. Uma vez que a renderização do “App” é suficiente para renderizar o componente “Remaining”, não se chegou a implementar esse `useEffect()`.

5. CONCLUSÃO

Consideramos que as dificuldades encontradas ao longo da elaboração do projeto foram ultrapassadas e que os objetivos do presente trabalho foram atingidos com sucesso, tendo sido aplicados e consolidados os conhecimentos adquiridos nas aulas de Linguagens Script.



**Instituto Superior
de Engenharia**

Politécnico de Coimbra